

Introducción a MongoDB

José R.R. Viqueira

- **Introducción**
- **Modelado de Datos**
- **Replicación**
- **Particionamiento (Sharding)**



■ Base de datos de documentos

■ Documento

- ▷ Estructura anidada compuesta de pares (campo, valor)
- ▷ Similar a objetos **JSON**
- ▷ Valores complejos

- Otros documentos anidados
- Arrays
- Arrays de documentos

```
{
  name: "sue",
  age: 26,
  status: "A",
  groups: [ "news", "sports" ]
}
```

← field: value
← field: value
← field: value
← field: value

■ Almacenamiento

- ▷ Colecciones de documentos
- ▷ vistas de solo lectura y vistas materializadas

■ Ventajas

- ▷ Correspondencia con estructuras usadas en lenguajes de programación
- ▷ Reducción de la necesidad de realizar **JOINS**
- ▷ **Esquema dinámico**
 - Proporciona polimorfismo fluido.



- Alto rendimiento en la persistencia de datos
 - ▷ Uso de un **modelo anidado**: Permite reducir el coste de E/S
 - ▷ **Indexación**: Permite usar claves de documentos anidados y arrays.
- Lenguaje de consulta expresivo
 - ▷ Operaciones de lectura y escritura (**CRUD**)
 - ▷ **Agregación**
 - ▷ Consultas **de texto completo y geoespaciales**.
- Alta disponibilidad
 - ▷ **Replica set**: Grupo de servidores con el mismo conjunto de datos
 - ▷ Recuperación automática ante fallos
 - ▷ Redundancia de datos
- Escalabilidad horizontal
 - ▷ Parte de su funcionalidad principal
 - ▷ El particionamiento (sharding) distribuye los datos entre máquinas
 - ▷ Se pueden crear zonas de datos (usando los valores de la clave)
- Varios motores de almacenamiento



- Durante el diseño del modelo de datos, tener en cuenta
 - ▷ Estructura inherente a los propios datos
 - ▷ Forma de acceder a los datos desde las aplicaciones
- **Flexibilidad en el esquema**
 - ▷ Las colecciones de MongoDB no necesitan tener un esquema declarado
 - _ No es necesario que tengan los mismos campos con los mismos tipos de dato.
 - _ Para modificar el esquema (añadir campos, etc.) solo es necesario realizar las modificaciones necesarias en los datos.
 - ▷ Cada documento se puede adaptar así a un objeto concreto, sin que todos los objetos tengan que ser idénticos en estructura
 - ▷ En la práctica los documentos de una colección serán similares
 - _ Se pueden realizar validaciones de esquema si es necesario



■ Estructura de los documentos

- ▷ Decisión de diseño clave
 - Definir la estructura de los documentos y la representación de las relaciones entre datos
- ▷ Datos Anidados
 - Los datos relacionado se almacenan en un único documento de estructura más compleja.
 - Un campo puede tener varias campos dentro o arrays

```
{
  _id: <ObjectId1>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```

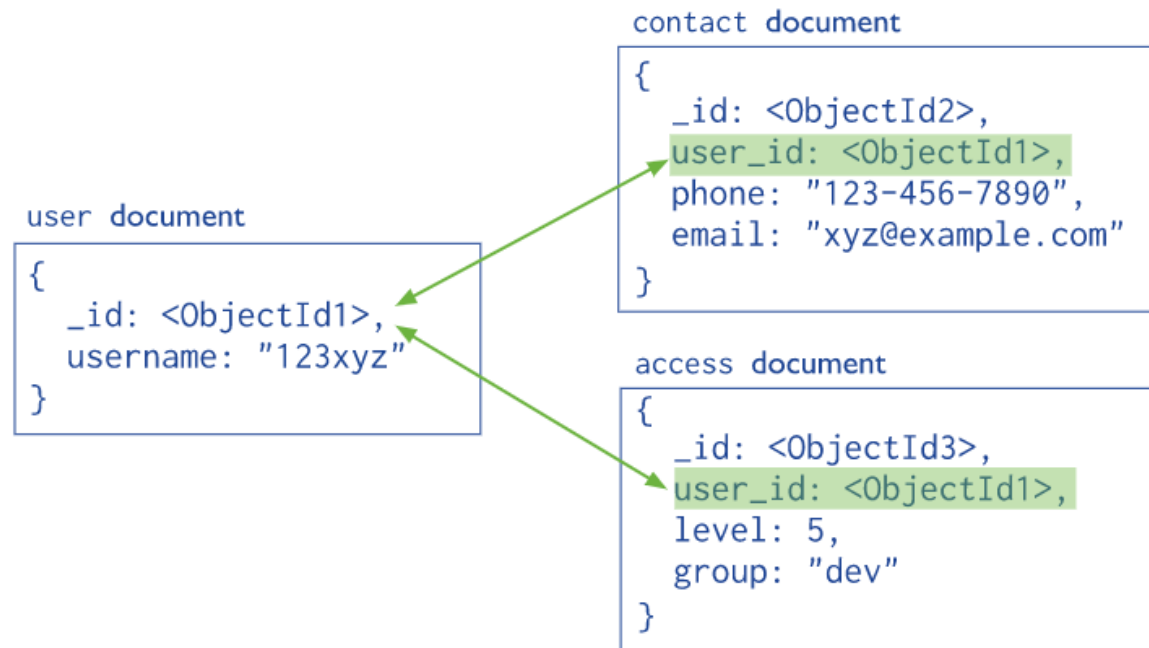
Embedded sub-document

Embedded sub-document

■ Estructura de los documentos

▷ Referencias

- También se pueden almacenar relaciones entre documentos con referencias.





■ **Atomicidad y operaciones de escritura**

▷ **Atomicidad a nivel de documento**

- Una operación de escritura es atómica a nivel de cada documento.
- El uso de los agregados simplifica la atomicidad
- Si una operación modifica varios documentos, la operación en su conjunto no es atómica
 - Se pueden entrelazar otras operaciones

▷ **Transacciones sobre varios documentos**

- Se pueden ejecutar cuando se necesita atomicidad al gestionar (leer y escribir) varios documentos

■ **Impacto de la estructura definida en el rendimiento**

- ▷ Considerar la forma de acceso durante el diseño.
- ▷ Usar también de forma apropiada la indexación

■ **Validación del esquema**

- ▷ Se pueden especificar reglas de validación al crear una colección
 - Se pueden aplicar de forma más o menos estricta y también generar errores o avisos (warnings)
- ▷ Soporte para JSON Schema desde la versión 3.6

Introducción

Modelado

Replicación

Sharding

■ Introducción

- ▷ *Replica set*: grupo de procesos **mongod** que mantienen el mismo conjunto de datos.
- ▷ Proporciona **redundancia** y **alta disponibilidad**

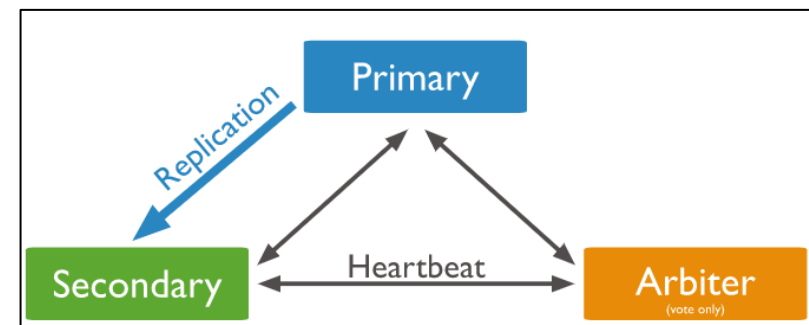
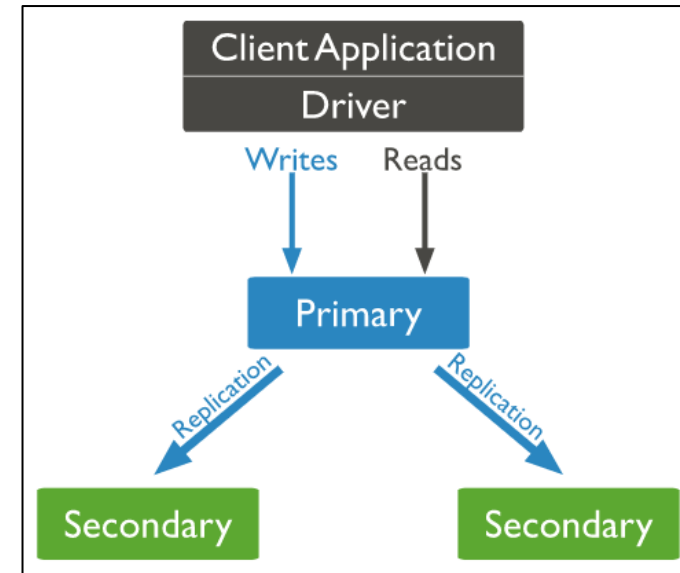


■ Redundancia y disponibilidad de los datos

- ▷ Varias copias de los mismos datos en servidores distintos
 - _ Tolerancia ante el fallo de algún servidor
- ▷ Puede mejorar el rendimiento de algunas lecturas
- ▷ Se puede mejorar la localidad de los datos y la disponibilidad de los mismos

■ Replicación en MongoDB

- ▷ En cada **replica set**
 - _ Un nodo primario
 - _ Varios nodos secundarios
 - _ Opcionalmente un nodo arbiter
- ▷ **Nodo primario**
 - _ Recibe todas las escrituras
- ▷ **Nodos Secundarios**
 - _ Replican los cambios que se van produciendo en el primario
 - _ Si el primario falla, uno de los secundarios puede ser elegido como primario
- ▷ **Nodo Arbiter**
 - _ Participa en las elecciones pero no tiene datos
 - _ No pueden transformarse en secundarios o primarios



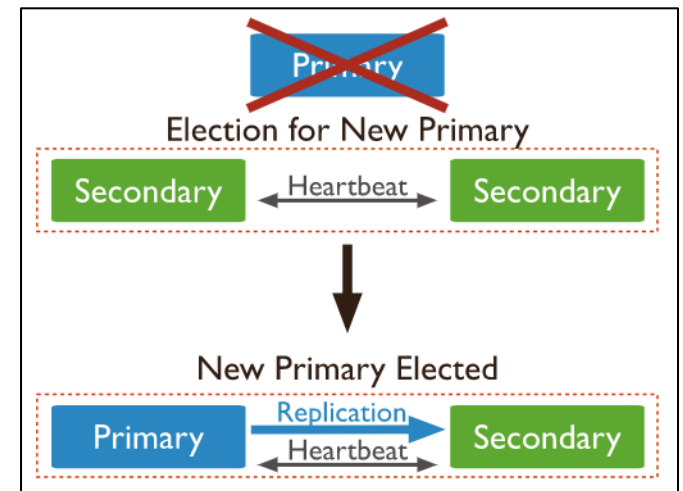


■ Replicación asíncrona

- ▷ Los nodos secundarios replican los cambios de los primarios de forma asíncrona.
- ▷ Existe cierto retraso entre que un cambio se hace en el primario y se replica en todos los secundarios
 - _ Para limitar este retraso se puede limitar el ratio de modificaciones que acepta el primario

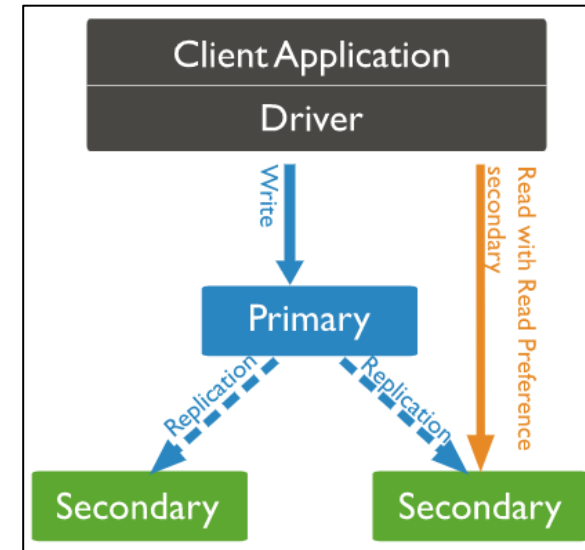
■ Recuperación automática ante fallos

- ▷ Cuando el primario no responde durante cierto tiempo, los secundarios eligen otro primario



■ Operaciones de lectura

- ▷ Por defecto se realizan del primario.
- ▷ Los clientes pueden indicar como preferencia de lectura hacerlo desde un secundario
- ▷ Los secundarios pueden no tener los últimos cambios del primario
- ▷ Las transacciones multi-documento deben de leer preferentemente del primario. Todas las operaciones deben de guiarse hacia el mismo miembro.
- ▷ Dependiendo del compromiso de lectura usado, los clientes pueden ver cambios antes de que estén confirmado (durables).
 - _ Si se usa el compromiso "local" o "available" se pueden leer datos con cambios que pueden todavía deshacerse.



Introducción

Modelado

Replicación



Sharding

■ Transacciones

- ▷ La preferencia de lectura debe de estar en el primario. Todas las operaciones deben de dirigirse al mismo nodo
- ▷ Las cambios no serán visibles hasta que se compromete
- ▷ Si se escriben varias particiones (shards), otro lector podría ver los cambios en una y no los cambios en otra.

■ Otras características

- ▷ Suscripción a flujos de cambios
- ▷ Se puede controlar la forma de elegir primarios
- ▷ Se pueden desplegar miembros en centros de datos distintos
- ▷ Se pueden dedicar miembros para backup, etc.

Introducción

Modelado

Replicación

Sharding



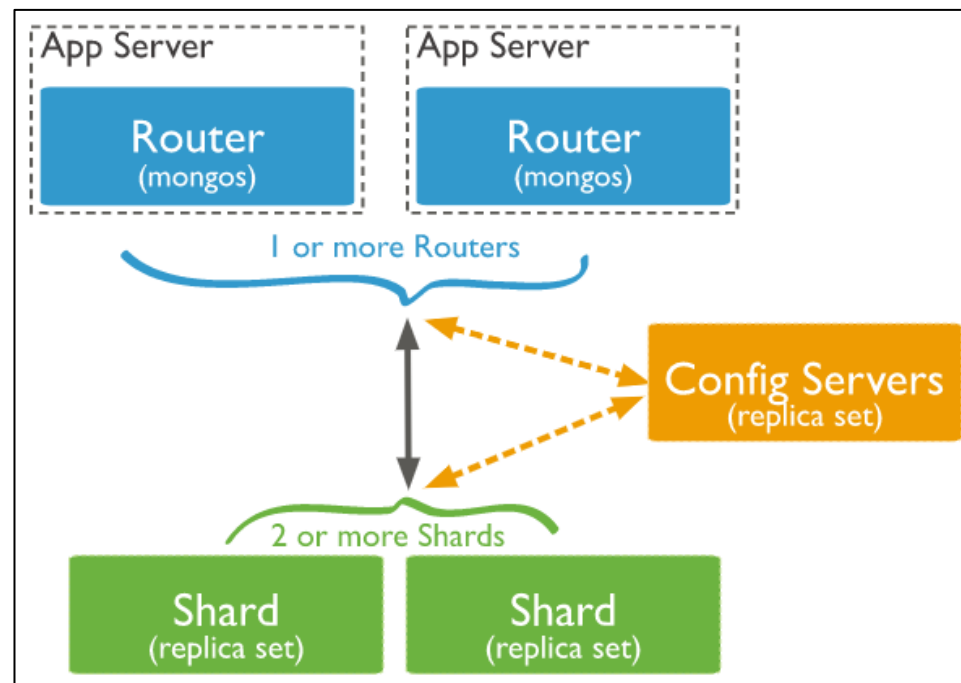
■ Introducción

- ▷ Distribución de los datos entre varias máquinas
 - _ Gran volumen de datos
 - _ Muchas operaciones por segundo
- ▷ **Escalamiento vertical**
 - _ Incrementar la capacidad de un único servidor (CPU, RAM, etc.).
 - _ Límite en las capacidades del hardware actual.
- ▷ **Escalamiento horizontal**
 - _ Particionar los conjuntos de datos para ser repartidos entre múltiples servidores.
 - _ Añadir nuevos servidores es en general más sencillo que mejorar el hardware de un servidor.
 - _ La contrapartida es la mayor complejidad de la infraestructura y del mantenimiento

■ Cluster particionado (Sharded Cluster) en MongoDB

▷ Componentes

- **Shards:** Puede ser un simple proceso o un replica set. Contiene los datos de una partición
- **mongos:** Enrutador de consultas. Interfaz entre los clientes y el cluster.
- **config server:** Almacenan los metadatos y la información de configuración del cluster



Introducción

Modelado

Replicación

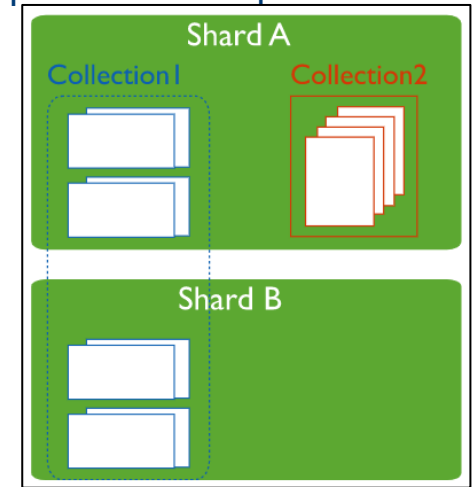
Sharding



- Clave de particionamiento (Shard key)
 - ▷ Un campo o varios campos del documento
 - ▷ Determina como se distribuyen los documentos en las particiones (shards)
 - ▷ Debe de existir un índice que empieza por la clave de particionamiento
 - ▷ La elección de la clave afecta mucho al rendimiento
- Chunks
 - ▷ Los datos se particionan en chunks.
- Balanceador
 - ▷ Para lograr una distribución uniforme de los chunks en el cluster un balanceador se ejecuta en segundo plano para mover chunks entre los shards
- Ventajas del particionamiento
 - ▷ Distribución de lecturas y escrituras entre los shards. Consultas sobre la clave se pueden dirigir a shards concretos
 - ▷ El espacio de almacenamiento se incrementa añadiendo nuevos shards
 - ▷ Alta disponibilidad al usar replica sets en cada componente (shards, config)

■ Consideraciones antes del particionamiento

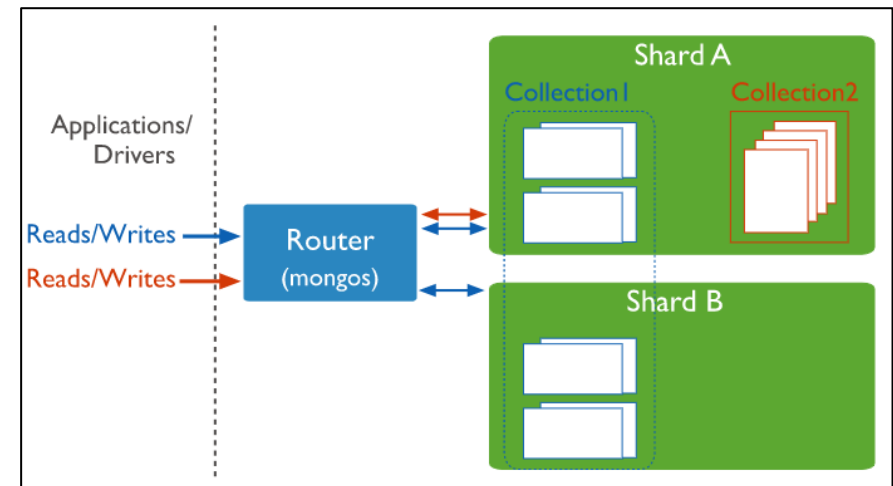
- ▷ La complejidad de la infraestructura demanda cierta planificación para la ejecución y el mantenimiento
- ▷ Una vez se particiona, no se puede desparticionar
- ▷ Hay que analizar con cuidado cual es la clave de particionamiento



■ Una base de datos puede tener colecciones particionadas y no particionadas

- ▷ Las no particionadas se almacenan en el shard primario

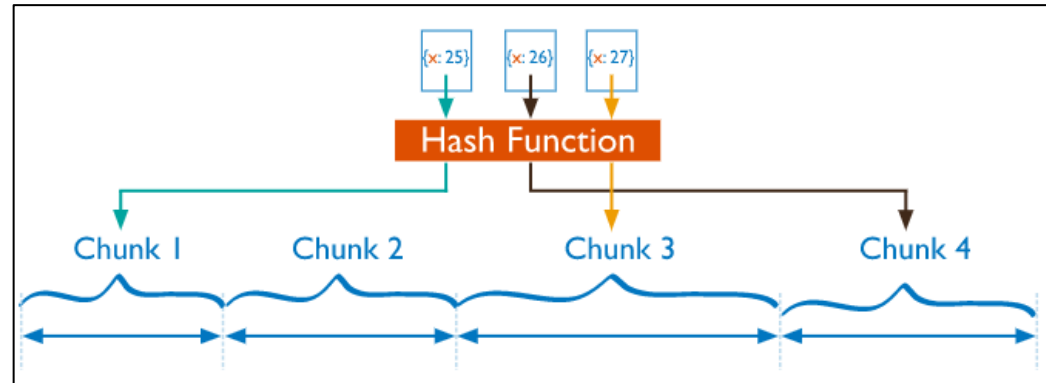
■ La conexión con un cluster particionado se hace a través de **mongos**



■ Estrategia de particionamiento

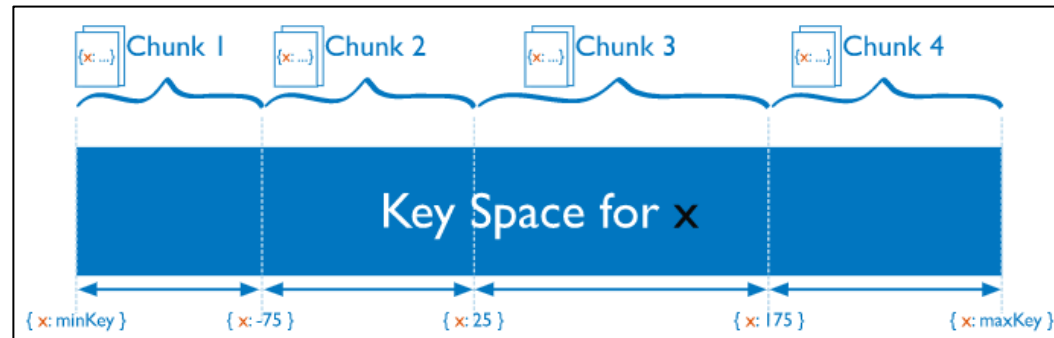
▷ Hashed Sharding

- Se evalúa una función hash sobre la clave. Cada chunk almacena documentos para un rango de valores resultantes del hash.
- Consiguen buena distribución sobre todo cuando se aplican sobre claves que cambian monótonicamente.
- Las búsquedas por rango pueden necesitar acceder va muchos chunks



▷ Ranged Sharding

- Los chunks se construyen sobre rangos de la clave
- Las búsquedas por rango se pueden dirigir a shards concretos
- La elección de la clave es muy importante para conseguir una buena distribución





■ Elección de la clave

▷ Cardinalidad

- Si elegimos una clave con pocos valores diferentes tendremos el problema de que el cluster podrá tener pocos shards

▷ Frecuencia

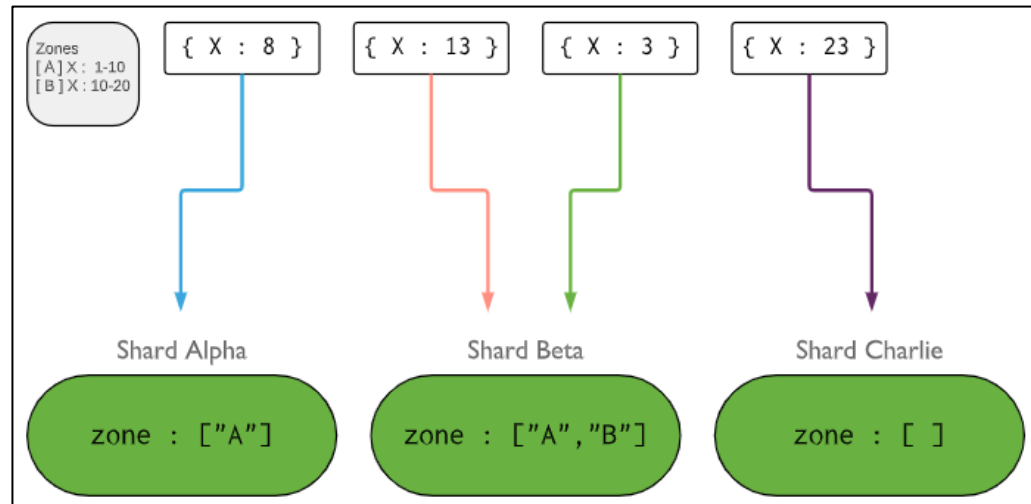
- La distribución de los valores de la clave influye.
- Si los valores están muy concentrados
 - los datos se almacenarían en unos pocos chunks
 - los accesos a los shards no estarían bien distribuidos, que es lo que se busca
- Lo ideal sería tener una distribución uniforme de las claves

▷ Monotonicidad

- Si la clave crece o decrece de forma monotónica
 - la mayoría de documentos irán a shards que almacenan valores extremos

■ Zonas en clusters particionados

- ▷ Cuando un cluster se expande por varios centros de datos las **zonas** pueden ayudar a mejorar la localidad de los datos
- ▷ Las zonas se crean en función de la clave de particionamiento
- ▷ Cada zona se puede asociar con uno o varias particiones (shards) del cluster
- ▷ El balanceador mueve chunks de una zona solo a shards de la misma zona



Introducción a MongoDB

<https://docs.mongodb.com/manual/>